HACKATHON ANIA · BOXLANG AI · CHALLENGE 06

Data Miner

Guía de pasos y requerimientos para terminar el proyecto

Qué instalar, qué cuentas crear, cómo correrlo, qué falta para cumplir los 4 requisitos del reto y cómo preparar la demo.

Mario · 11 de octubre de 2026 · BoxLang 1.18.0 · bx-ai 3.6.0

DÓNDE ESTAMOS

El código ya existe y funciona en modo demo, con los 4 requisitos del reto cubiertos. Lo que falta es correrlo en tu computadora, conectarlo a un modelo de IA real, validar con él la salida estructurada y ensayar la demo. Esta guía lo ordena en 9 fases con casillas para ir marcando.

Fase	Qué haces	Tiempo aprox.	¿Obligatoria?
0	Reunir cuentas y software	20 min	Sí
1	Instalar Java 21, BoxLang y bx-ai	15 min	Sí
2	Bajar el código y ubicar los jars	5 min	Sí
3	Correrlo en modo demo (sin IA)	10 min	Sí
4	Conectar un LLM real (Gemini o Groq)	1–2 h	Sí
5	Google Drive y Kaggle	30 min	Opcional
6	Base de datos por MCP	30 min	Opcional
7	Revisar anonimización, structured output, módulo, stock y ventas	45 min	Ya construido; solo validar
8	Preparar y ensayar la demo	1–2 h	Sí
9	Entregar	30 min	Sí

1. Los 4 requisitos del reto y cómo vamos

Texto del reto 06 «Data Miner» (PDF oficial del hackathon): construir un analista de datos autónomo que inspeccione un dataset o base de datos desconocidos y descubra información útil; que entienda los datos, formule preguntas, ejecute consultas, analice resultados y distinga los hechos de la interpretación de la IA.

#	Requisito	Estado	Qué falta
1	**Core.** Responder preguntas en lenguaje natural y descubrir solo al menos un hallazgo útil.	✓ **Cumplido**	Comprobarlo con un LLM real (fase 4). Hoy solo está probado con respuestas simuladas.
2	**Go further.** Agentes especializados de SQL, estadística, visualización o verificación.	✓ **Cumplido**	Nada. Coordinador + Datos + Analista + Científico ML, y la verificación re-ejecuta el SQL de cada hecho.
3	**Community goal.** Tools de base de datos, agentes SQL, skills de análisis, conectores o utilidades de consulta segura reutilizables.	✓ **Cumplido**	Módulo **bx-datatools** en `modulo/`: anonimización, SQL de solo lectura, calidad y conector MCP, con README y ejemplo que corre sin key.
4	**Potential BoxLang AI.** Agents, tools, structured output, pipelines, memory.	✓ **5 de 5**	Agents, tools, pipelines, memory y **structured output** (paso del pipeline). Validar la salida estructurada con Gemini (fase 4).

> **Prioridad si te falta tiempo:** fase 4 (LLM real) > fase 8 (demo ensayada) > fase 7 (revisar lo nuevo) > fase 6 (MCP) > fase 5 (Drive/Kaggle). Sin la fase 4 no hay demo.

Qué hace hoy el proyecto

- **Anonimiza al entrar** DUI, NIT, nombres, teléfonos, correos, direcciones y fechas de nacimiento, antes de que la IA vea nada.
- Recibe archivos CSV, TSV, Excel (varias hojas), JSON y ZIP, o los baja de Google Drive, Sheets o Kaggle.
- Da un diagnóstico de calidad inmediato, sin IA (puntaje 0–100 y problemas con su recomendación).
- Un coordinador reparte la misión entre 3 agentes: **Datos** (adquisición, perfil, limpieza, esquema, consejos), **Analista** (SQL, base remota por MCP, estadística, finanzas, gráficas) y **Científico ML** (7 tipos de modelo con Tribuo).
- Separa **hechos** (con su SQL, verificados ✓/✗) de **interpretaciones**, y genera un informe HTML descargable.

Fase 0. Cuentas y software que necesitas

Software

Pieza	Versión	Para qué	Cómo verificar
Java (JDK)	21	BoxLang corre sobre la JVM	`java -version`
BoxLang	1.18.0 o mayor	Lenguaje, `boxlang` y `boxlang-miniserver`	`boxlang --version`
Módulo bx-ai	3.6.0 o mayor	Agentes, tools, memoria, pipelines	`install-bx-module bx-ai`
Navegador	Chrome, Edge o Firefox	Ver la app y el informe	—
Python 3	3.10+	Solo para MCP y los scripts de prueba	`python3 --version`
Node.js	18+ (trae npx)	Solo para el puente MCP (supergateway)	`npx --version`
Git	cualquiera	Subir el proyecto a GitHub para entregarlo	`git --version`

Cuentas (todas gratis)

Cuenta	Dónde	Qué obtienes	¿Necesaria?
Google AI Studio (Gemini)	aistudio.google.com → Get API key	`GEMINI_API_KEY`	Sí (o Groq)
Groq (alternativa)	console.groq.com → API Keys	`GROQ_API_KEY`	Plan B
Kaggle	kaggle.com/settings → Create New Token	`kaggle.json` con usuario y key	Opcional
GitHub	github.com	Repositorio para entregar	Sí

Hardware

- Cualquier laptop con 8 GB de RAM. La app usa unos 500 MB; el ML con Tribuo tarda segundos con datasets de miles de filas.
- Internet para el LLM. Sin internet solo funciona el modo demo (mock), que es tu plan B.
- Ollama (modelo local) es posible pero no recomendado: en una laptop normal los modelos pequeños fallan más con tool calling.

> **Privacidad.** En el plan gratuito de Gemini, Google puede usar lo que envías para mejorar sus modelos. Para la demo usa datos sintéticos o públicos (los de `data/` y `pruebas/` ya lo son). Nunca subas datos reales de clientes.

Checklist fase 0

- ☐ Tengo Java 21 instalado.
- ☐ Tengo mi API key de Gemini (y opcionalmente la de Groq).
- ☐ Bajé mi kaggle.json.

☐ Tengo cuenta de GitHub.

Fase 1. Instalar BoxLang y bx-ai

Si no tienes Java 21, instálalo primero (por ejemplo Eclipse Temurin 21 desde adoptium.net). Luego:

Windows (PowerShell como Administrador)

```
powershell -NoExit -Command "iex ((New-Object System.Net.WebClient).DownloadString('https://install-windows.boxlang.io'))"
```

Mac o Linux

```
curl -fsSL https://install.boxlang.io/ | bash
# o en Mac con Homebrew:
brew tap ortus-boxlang/boxlang && brew install ortus-boxlang/boxlang/boxlang && install-boxlang
```

Después, en cualquier sistema (abre una terminal nueva para que tome el PATH):

```
boxlang --version           # debe decir 1.18.0 o mayor
install-bx-module bx-ai     # instala BoxLang AI de forma global
boxlang                     # REPL; escribe  println("hola")  y sal con Ctrl+C
```

Comandos que quedan instalados: `boxlang` (scripts y REPL), `boxlang-miniserver` (servidor web) e `install-bx-module` (módulos).

Checklist fase 1

☐ `boxlang --version` responde.

☐ `install-bx-module bx-ai` terminó sin errores.

Fase 2. Bajar el código

1. Descarga `boxlang-proyecto.zip` del hilo del proyecto y descomprímelo, por ejemplo en `Documentos/hackathon`.
2. La carpeta que importa es `data-miner/`:

```
data-miner/
├── README.md            instrucciones y trampas de BoxLang
├── miniserver.json      { "port": 8080, "webRoot": "./www" }
├── demo-equipo.bxs      demo por consola (sin web)
├── models/              Equipo, Catalogo, Fuentes, IngenieroDatos,
│                        Estadistica, ML, Informe, ConectorMCP
├── www/                 index.html, Application.bx, api/*.bxm
├── data/                creditos.csv, precios.csv (ejemplos)
├── mcp/                 base SQLite de ejemplo + script del servidor MCP
├── pruebas/             archivos "sucios" y un Kaggle falso
└── lib/                 40 jars: Tribuo (ML) y Apache POI (Excel)
```

3. **No borres** `lib/`**.** La web carga esos jars sola (`this.javaSettings` en `www/Application.bx`). Para la demo por consola, cópialos también a la carpeta de librerías de BoxLang:

```
# Mac / Linux
mkdir -p ~/.boxlang/lib && cp lib/*.jar ~/.boxlang/lib/
# Windows (PowerShell)
New-Item -ItemType Directory -Force "$HOME\.boxlang\lib"; Copy-Item lib\*.jar "$HOME\.boxlang\lib\"
```

Checklist fase 2

- ☐ Tengo la carpeta data-miner/ con lib/ (40 jars).
- ☐ Copié los jars a ~/.boxlang/lib/.

Fase 3. Correrlo en modo demo (sin IA)

Sin ninguna variable configurada, el proyecto usa el proveedor `mock` de bx-ai: los agentes reciben respuestas guionizadas pero **las tools sí se ejecutan de verdad** (SQL, estadística, ML, gráficas). Sirve para comprobar que tu instalación está bien.

Web

```
cd data-miner
boxlang-miniserver          # abre http://localhost:8080
```

1. En la página, sube `pruebas/fuentes/ventas_erp.xlsx`. Debes ver 2 tablas y su diagnóstico de calidad (la de ventas sale con puntaje bajo, unos 33).
2. Escribe una misión, por ejemplo «¿Qué explica el impago?», y pulsa enviar. Debes ver la traza del equipo, hechos con ✓, gráficas y el informe.
3. La tabla `clientes_sv_clientes` ya viene cargada y anonimizada; la respuesta muestra un recuadro de Privacidad.
4. Pulsa **Descargar informe** y ábrelo: tiene que mostrarse completo aunque apagues el wifi.

Consola

```
boxlang demo-equipo.bxs     # sin Kaggle ni MCP algunas tools darán error controlado
```

Para la demo de consola completa (Kaggle falso y MCP) mira la sección «Correr» del README.

Si algo falla

Síntoma	Causa probable	Arreglo
Puerto 8080 ocupado	Otra app usa el puerto	Cambia "port" en miniserver.json
Error de clase Tribuo o POI	No encontró los jars	Revisa lib/ y, en consola, ~/.boxlang/lib/
aiAgent no existe	Falta el módulo bx-ai	`install-bx-module bx-ai`
Cambios en .bx no se ven	Caché de la aplicación	Abre la página con `?reload=1`
«Abre primero la página»	No hay sesión todavía	Carga http://localhost:8080 antes de subir

Checklist fase 3

- ☐ La web abre y el diagnóstico del Excel aparece.
- ☐ Una misión en modo demo termina con informe.

Fase 4. Conectar un LLM real (lo más importante)

El proyecto lee estas variables de entorno al arrancar (`www/Application.bx`):

Variable	Valor de ejemplo	Para qué
`AI_PROVIDER`	`gemini` o `groq`	Proveedor. Sin ella = mock
`GEMINI_API_KEY`	tu key	La detecta bx-ai sola (para Groq: `GROQ_API_KEY`)
`AI_MODEL`	un modelo "flash" o "pro" actual	Modelo principal. Revisa la lista vigente en AI Studio
`AI_MODEL_LIGERO`	un modelo más chico	Opcional: para el agente Datos. Cada modelo tiene su propia cuota
`EQUIPO_MODO`	`compacto`	compacto (3 agentes, por defecto) o completo (7)

Windows (PowerShell, en la misma ventana donde arrancas el servidor)

```
$env:AI_PROVIDER="gemini"
$env:GEMINI_API_KEY="tu_key"
$env:AI_MODEL="nombre-del-modelo"
boxlang-miniserver
```

Mac o Linux

```
export AI_PROVIDER=gemini GEMINI_API_KEY=tu_key AI_MODEL=nombre-del-modelo
boxlang-miniserver
```

Cómo probarlo, de menos a más

1. **Prueba mínima:** «¿Qué tablas hay y cuántas filas tiene cada una?». Gasta pocas llamadas y confirma que la key y el tool calling funcionan.
2. **Una sola especialidad:** sube el Excel y pide «Revisa la calidad de ventas y límpiala». Debe responder el agente Datos y crear `ventas_erp_ventas_limpia`.
3. **Misión completa:** «¿Qué explica el impago en creditos? Entrena un modelo». Unas 18 llamadas al LLM.
4. Revisa en cada respuesta: que los hechos traigan SQL y salgan ✓, que no invente columnas, y que el informe tenga gráficas.

Problemas esperables y qué hacer

Problema	Qué hacer
Error 429 (cuota)	Ya hay reintentos automáticos (8 s, 16 s, 32 s…). Si persiste: modo compacto, `AI_MODEL_LIGERO`, misiones más enfocadas o cambiar a Groq.
El coordinador responde texto en vez de JSON	La función `parsear` de `models/Equipo.bx` lo tolera; si se ve mal, refuerza el formato en las instrucciones del coordinador o haz la fase 7.2.
El modelo inventa columnas o tablas	Pídele en las instrucciones que siempre llame a `listar_tablas` o `perfilar_tabla` antes de consultar.
Tool calling falla con el modelo elegido	Prueba otro modelo del mismo proveedor; los más chicos fallan más.
Tarda mucho	Cada agente hace varias llamadas. Con ~10 req/min una misión completa tarda unos 3 min.

> **Ajusta los prompts aquí:** las instrucciones de cada agente están en `models/Equipo.bx` (campo `instructions`). Cambia una cosa a la vez y repite la misma misión para comparar.

Checklist fase 4

- ☐ La prueba mínima responde con un LLM real.
- ☐ La misión completa termina con hechos ✓ e informe.
- ☐ Anoté qué modelo funcionó mejor y cuántos minutos tarda.

Fase 5. Google Drive y Kaggle (opcional)

Para una empresa salvadoreña la fuente principal serán sus propios Excel, CSV y su base de datos. Drive sirve para hojas compartidas; Kaggle queda solo para explorar datos públicos.

Kaggle

1. En kaggle.com/settings pulsa **Create New Token**; se descarga `kaggle.json`.
2. Ponlo en `~/.kaggle/kaggle.json` (Windows: `C:\Users\TU_USUARIO\.kaggle\kaggle.json`), o usa variables:

```
export KAGGLE_USERNAME=tu_usuario KAGGLE_KEY=tu_key        # Mac/Linux
$env:KAGGLE_USERNAME="tu_usuario"; $env:KAGGLE_KEY="tu_key"  # Windows
```

3. Prueba la misión: «Busca en Kaggle un dataset de ventas de supermercado, descárgalo y dime qué sucursal vende más».
4. Si falla: revisa que el dataset sea público y pequeño (menos de unos 50 MB) y que el token sea el más reciente.

Google Drive y Sheets

- Solo links **públicos** («Cualquier persona con el enlace»). Los privados necesitan OAuth y no están implementados.
- Sheets se baja como Excel con todas sus hojas; Drive detecta el tipo de archivo por el contenido.
- Prueba: «Descarga este archivo <link> y revisa su calidad».

Checklist fase 5

- ☐ Una descarga real de Kaggle funcionó.
- ☐ Un link público de Drive o Sheets funcionó.

Fase 6. Base de datos por MCP (opcional)

Muestra que el agente puede consultar una base remota de forma segura: solo recibe 3 tools de lectura y nunca las de escritura. Requiere Python 3 y Node. En Windows conviene usar WSL o Git Bash, porque el script es de bash.

```
cd data-miner
bash mcp/iniciar_mcp_sqlite.sh                # crea mcp/tienda.db y la sirve en :8766
# en otra terminal, antes de arrancar el servidor web:
export MCP_DB_URL=http://localhost:8766/mcp
boxlang-miniserver
```

- El script instala `mcp-server-sqlite` con versiones fijadas (`mcp==1.6.0`, `pydantic==2.10.6`); con versiones nuevas no arranca.
- Prueba: «¿Qué productos de la base de la tienda venden más?». El Analista debe usar `db_consultar`.
- Para usar tu propia base SQLite: `bash mcp/iniciar_mcp_sqlite.sh /ruta/a/tu.db 8766`.

Checklist fase 6

☐ El Analista consulta la base por MCP.

Fase 7. Revisar lo que ya se construyó para los requisitos

Estas cuatro piezas ya están hechas y probadas en modo demo. Solo tienes que verlas funcionar en tu máquina y, con el LLM real, confirmar la 7.2.

7.1 Anonimización de los datos entrantes (primer frente de confianza)

- Todo archivo pasa por `models/Anonimizador.bx` antes de ser tabla: DUI, NIT, nombres, teléfonos, correos y tarjetas a pseudónimos estables; dirección a departamento; fecha de nacimiento a rango de edad; datos personales dentro de comentarios, enmascarados. Las respuestas de MCP también.
- Prueba sin IA: `boxlang probar-anonimizacion.bxs`. En la web, sube `pruebas/fuentes/clientes_sv.xlsx` y mira el aviso de privacidad.
- Variables: `ANONIMIZAR=no` la apaga (no lo hagas con datos reales); `ANON_SALT` fija la sal para que los pseudónimos no cambien entre reinicios.
- Límite: es por reglas. Con datos nuevos, revisa que el aviso de privacidad liste todas las columnas personales.

7.2 Requisito 4: structured output

- El pipeline es `coordinador → estructurar → verificar`. `estructurar` usa `modelo.structuredOutput( esquema )` de bx-ai, sin tools, y si falla cae a leer el texto.
- **Con Gemini:** corre una misión y mira el encabezado de la respuesta en la web. Debe decir «structured output». Si dice «parseo de texto», copia el mensaje de error y ajusta el modelo o el prompt de `estructurar()` en `models/Equipo.bx`.

7.3 Requisito 3: módulo bx-datatools

- Carpeta `modulo/bx-datatools/`: módulo de BoxLang con `DataTools.bx` (catálogo + tools para cualquier `aiAgent`), README y `ejemplos/analista.bxs`, que corre sin key.
- Instalar y probar: `cp -r modulo/bx-datatools ~/.boxlang/modules/bxdatatools` y luego `boxlang modulo/bx-datatools/ejemplos/analista.bxs`.
- Si cambias clases en `models/`, corre `bash modulo/empaquetar.sh` para copiarlas al módulo.
- Para la entrega, súbelo a GitHub como repositorio aparte (licencia Apache 2.0) y enlázalo como el aporte a la comunidad.

7.4 Operaciones: cuándo se agota el stock y qué ventas priorizar

- Dos tools nuevas en `models/Operaciones.bx`. El LLM solo elige la tool y explica: los números salen de fórmulas fijas y quedan en tablas que se verifican con SQL.
- `pronostico_stock`: demanda de los últimos 14 días + tendencia, stock de seguridad, punto de reorden, fecha de quiebre y cuánto pedir por producto. Estado agotado / crítico / pronto / ok.

- `priorizar_ventas`: regresión logística entrenada con las oportunidades ya ganadas o perdidas; ordena las abiertas por probabilidad × monto × margen y sugiere una acción.
- Prueba en la web con los chips «¿Qué productos se van a agotar y cuánto debo pedir?» y «¿Qué oportunidades de venta debo priorizar esta semana?» (datos de `pruebas/fuentes/operaciones_sv.xlsx`). Con Gemini, confirma que el agente elige la tool correcta.
- Para datos reales, el cliente sube un Excel con hojas parecidas: ventas por día y SKU, inventario con stock y días de entrega, y oportunidades con estado, monto y margen.

7.5 Mejoras opcionales si sobra tiempo

- Mostrar en la UI qué agente está trabajando en tiempo real (hoy se ve la traza al final).
- Un botón «Limpiar tabla» en la tarjeta de calidad que llame a `limpiar_tabla` sin IA.

Checklist fase 7

☐ Vi el aviso de privacidad al subir clientes_sv.xlsx.

☐ Con Gemini, la respuesta dice «structured output».

☐ El ejemplo del módulo corre en mi máquina.

☐ Con Gemini, las misiones de stock y de priorización dan hechos ✓.

Fase 8. Preparar y ensayar la demo

Guion sugerido (4 a 5 minutos)

Min.	Qué muestras	Qué dices
0:00	Portada o la app vacía	El problema: una empresa mediana tiene datos en Excel sueltos y nadie los analiza.
0:30	Subes `clientes_sv.xlsx`	Primero la privacidad: DUI, NIT, nombres y teléfonos se anonimizan antes de que la IA los vea.
1:00	Subes `ventas_erp.xlsx`	Diagnóstico inmediato sin IA: duplicados, «Norte/norte/NORTE», montos como texto. Puntaje 33.
1:30	Misión: «revisa y limpia ventas»	El agente Datos limpia: la tabla limpia sube a 62. La original no se toca.
2:00	Misión: «¿qué segmento compra más?» y «¿qué explica el impago?»	El coordinador reparte. El Analista une clientes y compras por el DUI anonimizado; el Científico ML compara modelos.
2:45	Misión: «¿qué productos se van a agotar?» y «¿qué ventas priorizar?»	No adivina: calcula con fórmulas fijas y un modelo entrenado con su historia. Dice qué pedir hoy y qué oportunidad cerrar primero.
3:30	Hechos ✓ e interpretaciones	Cada hecho trae su SQL y se re-ejecuta. Detecta la fuga de datos de `dias_mora_max`.
4:00	Informe descargado	Funciona sin internet y se imprime a PDF.
4:30	Código de `Equipo.bx`	Agents, subAgents, tools, memory, pipeline y structured output: los 4 requisitos.

Antes de presentar

☐ Ensayé el guion completo al menos 2 veces con el LLM real y medí el tiempo.

☐ Tengo la app abierta y ya «calentada» (una misión corrida antes) para no esperar la carga inicial.

☐ Tengo capturas y un informe HTML ya generado por si algo falla en vivo.

☐ **Plan B listo:** otra terminal con `AI_PROVIDER` sin definir (mock) para mostrar el flujo sin internet ni cuota.

☐ Cerré otras apps que usen la red y tengo el celular como hotspot de respaldo.

☐ Sé responder: ¿qué pasa si el modelo se equivoca? (la verificación re-ejecuta el SQL y marca ✗); ¿es seguro? (solo SELECT, sin tools de escritura); ¿cuánto cuesta? (todo open source, solo la cuota del LLM).

Fase 9. Entregar

1. Sube `data-miner/` a un repositorio de GitHub. Los jars de `lib/` pesan 26 MB y caben; si prefieres no subirlos, explica en el README de dónde bajarlos.
2. **No subas keys.** Revisa que ningún archivo tenga tu `GEMINI_API_KEY` ni tu `kaggle.json`.
3. Actualiza la tabla «Qué está probado» del README con lo que probaste en las fases 4 a 6.
4. Adjunta la propuesta (`data-miner-propuesta.pdf`), capturas y, si lo piden, un video corto de la demo.
5. Enlaza el módulo bx-datatools (fase 7.3) como el aporte a la comunidad.

Checklist final

Una página para imprimir y marcar.

☐	Paso	Fase
☐	Java 21, BoxLang 1.18+ y bx-ai instalados	1
☐	Proyecto descomprimido, jars en lib/ y en ~/.boxlang/lib/	2
☐	Modo demo: web abre, Excel se diagnostica, misión con informe	3
☐	LLM real: prueba mínima, una especialidad y misión completa	4
☐	Prompts ajustados y modelo elegido	4
☐	Link público de Drive o Sheets (opcional)	5
☐	Base de datos por MCP (opcional)	6
☐	Anonimización vista en la web con clientes_sv.xlsx	7.1
☐	Structured output confirmado con Gemini	7.2
☐	Ejemplo del módulo bx-datatools corriendo	7.3
☐	Misiones de stock y priorización con hechos ✓	7.4
☐	Guion ensayado 2 veces, con tiempo medido	8
☐	Plan B en modo mock y capturas listas	8
☐	Repositorio en GitHub sin keys, README actualizado	9
☐	Propuesta, capturas y video entregados	9

Referencias

- README del proyecto: `data-miner/README.md` (comandos, costos del plan gratuito, trampas de BoxLang).
- Guía de BoxLang del hackathon: `guia-boxlang-hackathon.md`.
- Documentación de BoxLang AI: ai.ortusbooks.com · Ejemplos: github.com/ortus-boxlang/bx-ai.
- Límites del plan gratuito de Gemini: aistudio.google.com/rate-limit.